Name: Ayushi Jaiswal

U.S.N: 23BTRCF007

College Name: Jain University

Batch: CSE - CTIS

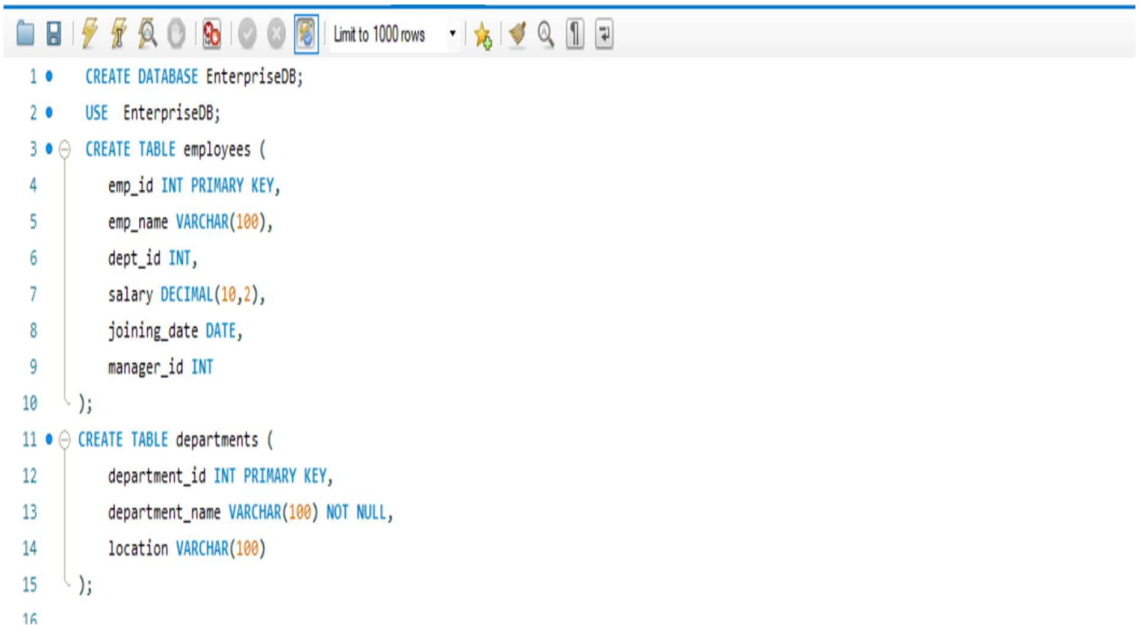
Date: 28/10/2026

ASSIGNMENT – 2

SECTION A: ADVANCED SQL

1. Database Design and Advanced Queries

- Creating Database named EnterpriseDB and using it
- Creating table for employees and departments

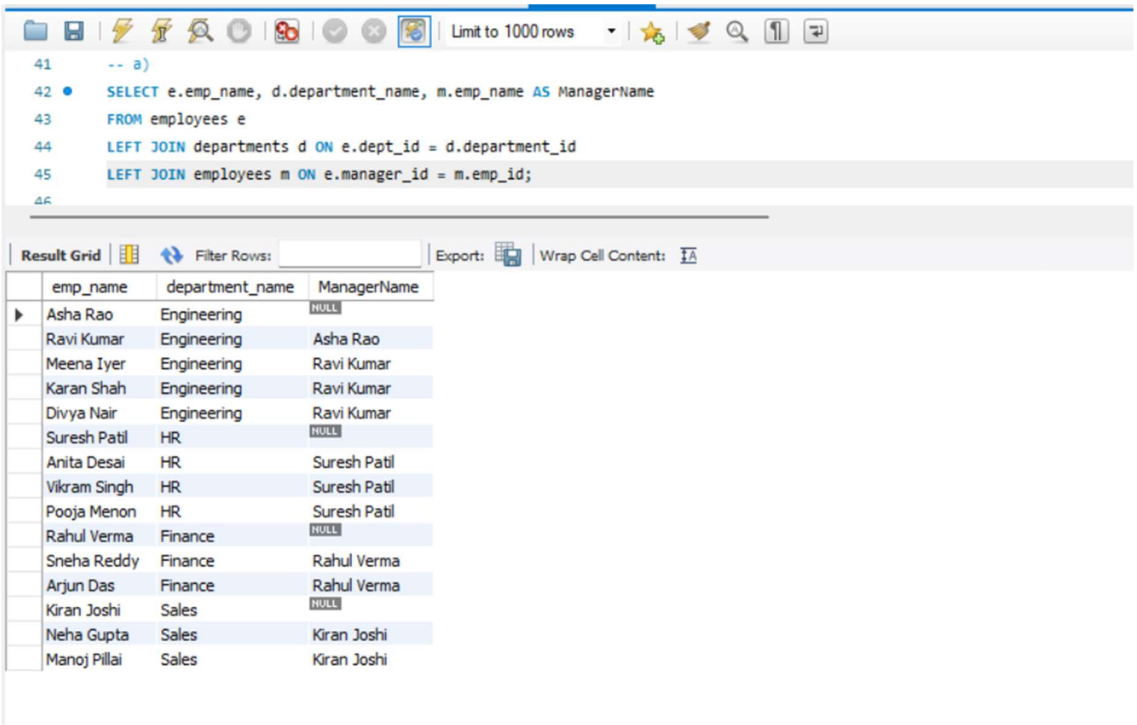


- Inserting Values for employees and departments Table

```sql
INSERT INTO employees VALUES
(1 ,'Asha Rao' ,10,150000,'2019-03-15',NULL),
(6 ,'Suresh Patil',20, 90000,'2018-11-20',NULL),
(10,'Rahul Verma' ,30,130000,'2017-08-25',NULL),
(13,'Kiran Joshi' ,40,100000,'2019-10-05',NULL),
(2 ,'Ravi Kumar'  ,10,120000,'2020-06-01',1),
(3 ,'Meena Iyer'  ,10, 95000,'2021-01-10',2),
(4 ,'Karan Shah'  ,10, 95000,'2022-07-18',2),
(5 ,'Divya Nair'  ,10, 70000,'2025-02-03',2),
(7 ,'Anita Desai' ,20, 60000,'2021-09-09',6),
(8 ,'Vikram Singh',20, 60000,'2023-04-12',6),
(9 ,'Pooja Menon' ,20, 45000,'2025-05-30',6),
(11,'Sneha Reddy' ,30, 85000,'2020-12-14',10),
(12,'Arjun Das'   ,30, 85000,'2024-03-01',10),
(14,'Neha Gupta'  ,40, 55000,'2025-08-19',13),
(15,'Manoj Pillai',40, 40000,'2025-11-02',13);

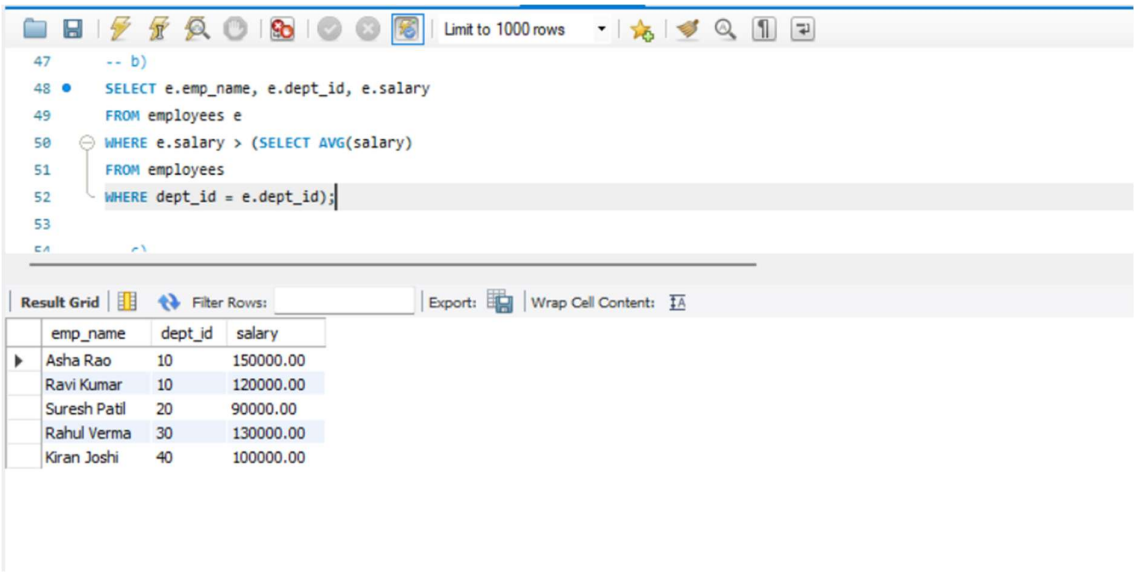
INSERT INTO departments VALUES
(10,'Engineering','Bengaluru'),
(20,'HR','Mysuru'),
(30,'Finance','Mumbai'),
(40,'Sales','Delhi'),
(50,'Marketing','Chennai');
```

1. a) Display employee names, department names, and manager names using joins.



emp_name	department_name	ManagerName
Asha Rao	Engineering	NULL
Ravi Kumar	Engineering	Asha Rao
Meena Iyer	Engineering	Ravi Kumar
Karan Shah	Engineering	Ravi Kumar
Divya Nair	Engineering	Ravi Kumar
Suresh Patil	HR	NULL
Anita Desai	HR	Suresh Patil
Vikram Singh	HR	Suresh Patil
Pooja Menon	HR	Suresh Patil
Rahul Verma	Finance	NULL
Sneha Reddy	Finance	Rahul Verma
Arjun Das	Finance	Rahul Verma
Kiran Joshi	Sales	NULL
Neha Gupta	Sales	Kiran Joshi
Manoj Pillai	Sales	Kiran Joshi

1. b) Find employees earning more than their department's average salary.

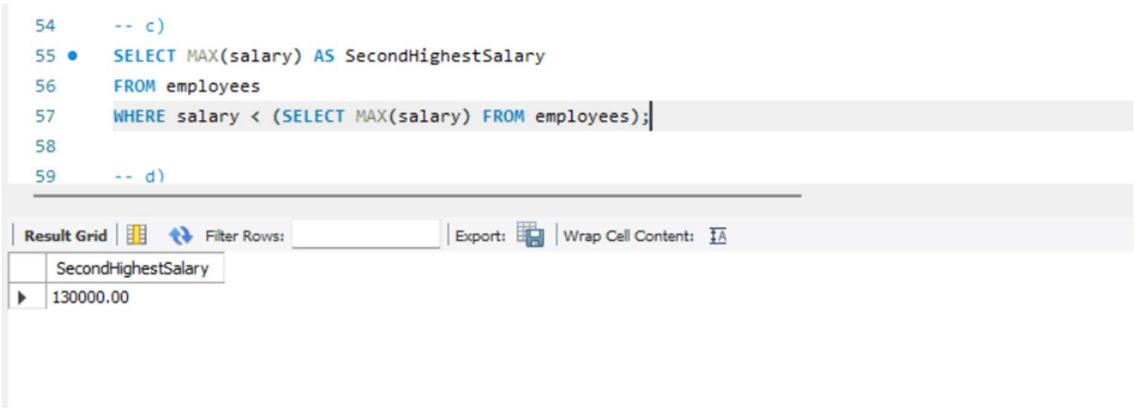


```
47    -- b)
48 •  SELECT e.emp_name, e.dept_id, e.salary
49    FROM employees e
50    WHERE e.salary > (SELECT AVG(salary)
51    FROM employees
52    WHERE dept_id = e.dept_id);
53
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	dept_id	salary
Asha Rao	10	150000.00
Ravi Kumar	10	120000.00
Suresh Patil	20	90000.00
Rahul Verma	30	130000.00
Kiran Joshi	40	100000.00

1. c) Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.



```
54    -- c)
55 •  SELECT MAX(salary) AS SecondHighestSalary
56    FROM employees
57    WHERE salary < (SELECT MAX(salary) FROM employees);
58
59    -- d)
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SecondHighestSalary
130000.00

1. d) Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.

```sql
-- d)
WITH RankedEmployees AS (
    SELECT
        e.emp_id,
        e.emp_name,
        d.department_name,
        e.salary,
        DENSE_RANK() OVER (
            PARTITION BY e.dept_id
            ORDER BY e.salary DESC
        ) AS salary_rank
    FROM employees e
    JOIN departments d
        ON e.dept_id = d.department_id
)
SELECT
    emp_id,
    emp_name,
    department_name,
    salary,
    salary_rank
FROM RankedEmployees
WHERE salary_rank <= 3
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	department_name	salary	salary_rank
1	Asha Rao	Engineering	150000.00	1
2	Ravi Kumar	Engineering	120000.00	2
3	Meena Iyer	Engineering	95000.00	3
4	Karan Shah	Engineering	95000.00	3
10	Rahul Verma	Finance	130000.00	1
11	Sneha Reddy	Finance	85000.00	2
12	Arjun Das	Finance	85000.00	2
6	Suresh Patil	HR	90000.00	1
7	Anita Desai	HR	60000.00	2
8	Vikram Singh	HR	60000.00	2
9	Pooja Menon	HR	45000.00	3
13	Kiran Joshi	Sales	100000.00	1
14	Neha Gupta	Sales	55000.00	2
15	Manoj Pillai	Sales	40000.00	3

2. Sales Analytics and Query Optimization

- Creating Table for sales and inserting values of 15 records

```sql
CREATE TABLE Sales (
SaleID      INT PRIMARY KEY,
CustomerID  INT NOT NULL,
SaleDate    DATE NOT NULL,
ProductName VARCHAR(50) NOT NULL,
Quantity INT NOT NULL,
UnitPrice DECIMAL(10,2) NOT NULL
);

INSERT INTO Sales VALUES
(1 ,101, '2026-01-05', 'Laptop', 2, 55000),
(2 ,102, '2026-01-12', 'Mouse', 10, 500),
(3 ,103, '2026-01-20', 'Keyboard', 5, 1500),
(4 ,101, '2026-02-02', 'Monitor', 3, 12000),
(5 ,104, '2026-02-10', 'Laptop', 1, 55000),
(6 ,105, '2026-02-18', 'Mouse', 20, 500),
(7 ,102, '2026-02-25', 'Keyboard', 4, 1500),
(8 ,106, '2026-03-03', 'Monitor', 2, 12000),
(9 ,103, '2026-03-09', 'Laptop', 3, 55000),
(10, 107, '2026-03-15', 'Headphones', 6, 2500),
(11, 104, '2026-03-28', 'Mouse', 15, 500),
(12, 105, '2026-04-04', 'Headphones', 4, 2500),
(13, 108, '2026-04-11', 'Laptop', 1, 55000),
(14, 101, '2026-04-19', 'Keyboard', 8, 1500),
(15, 106, '2026-04-27', 'Monitor', 5, 12000);
```

2. a) Calculate total revenue for each product.

```sql
SELECT ProductName, SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName
ORDER BY TotalRevenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	TotalRevenue
Laptop	385000.00
Monitor	120000.00
Keyboard	25500.00
Headphones	25000.00
Mouse	22500.00

2. b) Calculate monthly revenue and month-over-month revenue change using a window function.

```sql
-- b)
WITH MonthlySales AS (
SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS Month,
SUM(Quantity * UnitPrice) AS Revenue
FROM Sales
GROUP BY DATE_FORMAT(SaleDate, '%Y-%m')
)
SELECT Month, Revenue,
Revenue - LAG(Revenue) OVER (ORDER BY Month) AS Revenue_Change
FROM MonthlySales
ORDER BY Month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Month	Revenue	Revenue_Change
2026-01	122500.00	NULL
2026-02	107000.00	-15500.00
2026-03	211500.00	104500.00
2026-04	137000.00	-74500.00

2. c) Rewrite the following query to improve the potential for index usage and explain why your version may perform better.

```sql
-- c)
SELECT *
FROM employees
WHERE joining_date >= '2025-01-01' AND joining_date < '2026-01-01';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	dept_id	salary	joining_date	manager_id
5	Divya Nair	10	70000.00	2025-02-03	2
9	Pooja Menon	20	45000.00	2025-05-30	6
14	Neha Gupta	40	55000.00	2025-08-19	13
15	Manoj Pillai	40	40000.00	2025-11-02	13
NULL	NULL	NULL	NULL	NULL	NULL

Using Year() function will need to check for each row since it can not use indexing whereas using range from'2025-01-01' to '2026-01-01' allows to check only for all the rows with 2025 which makes it more optimized and more accurate result.

3. SQL Troubleshooting

3. a) Explain how an execution plan helps identify expensive operations in a query.

- An execution plan shows the operations the database uses to execute a query, such as scans, seeks, joins, sorts, and aggregations. Estimated and actual row counts, operator costs, and warnings can reveal expensive scans, bad cardinality estimates, missing or ineffective indexes, spills, and inefficient join choices. Compare actual versus estimated rows and optimize the largest genuine bottleneck; the displayed cost is an estimate, not a direct measure of elapsed time.

3. b) Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning.

- CREATE INDEX IX_Employees_DepartmentID_Salary

  ON Employees (DepartmentID, Salary);

A composite index can help queries filtering by DepartmentID and Salary, especially when DepartmentID is commonly used first and Salary is used as a range/filter. Column order should match real query patterns and selectivity. If queries frequently filter by Salary alone, a separate salary-leading index may be considered after measuring workload. Indexes improve reads but consume storage and add write/update overhead.

3. c) Write a query to identify duplicate customer email addresses and display only addresses occurring more than once.

```
SELECT EmailAddress, COUNT(*) AS Occurrences
FROM Customers
WHERE EmailAddress IS NOT NULL
GROUP BY EmailAddress
HAVING COUNT(*) > 1;
```

SECTION B: LINUX

4. Log Analysis

Assume /var/log/application.log contains millions of log entries in the following format: 2026-10-28 10:15:23 ERROR payment-service Transaction failed.

Write Linux commands to:

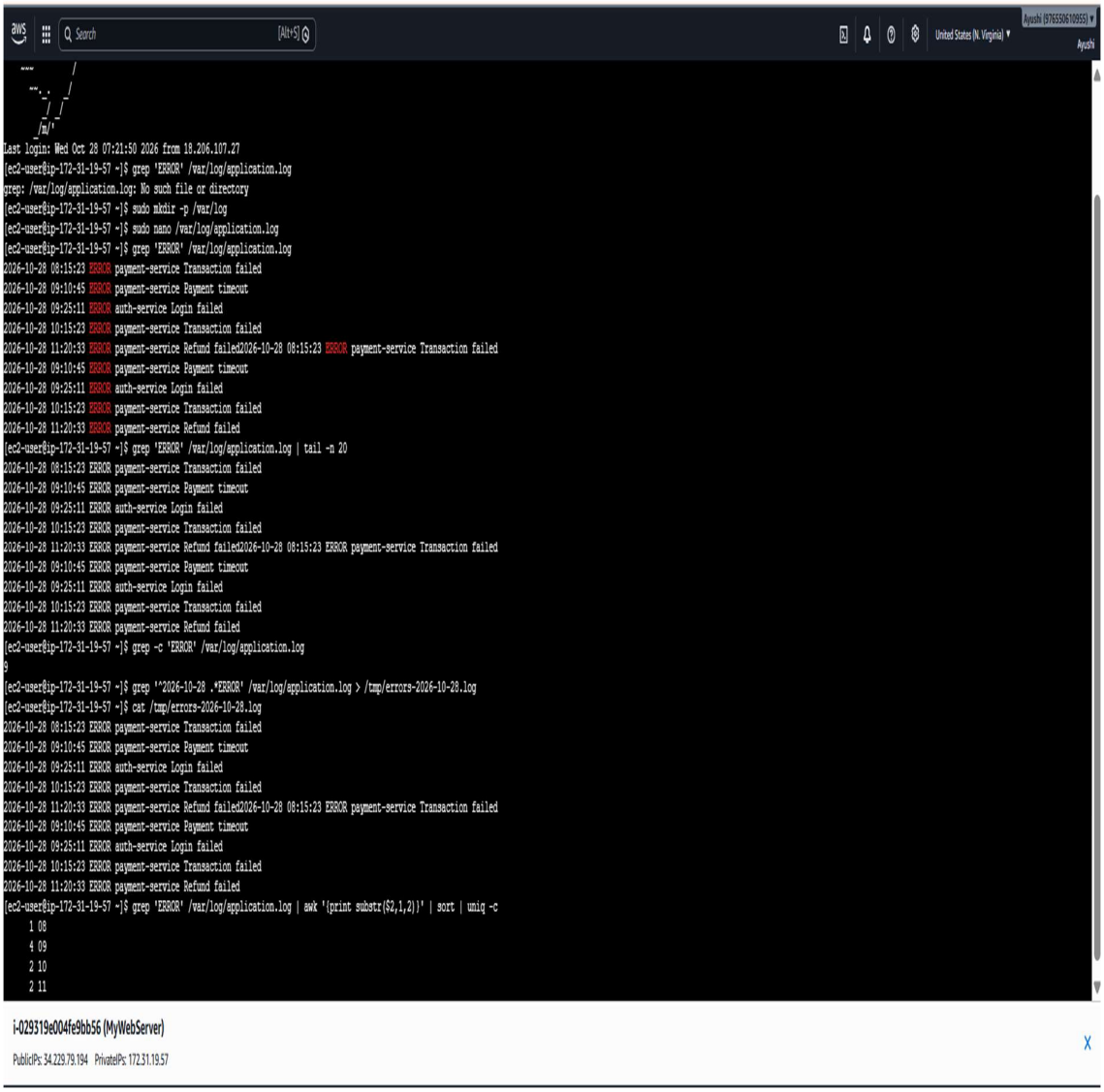
4. a) Find all lines containing ERROR.

4. b) Display the 20 most recent error entries in the file.

4. c) Count the total number of error entries.

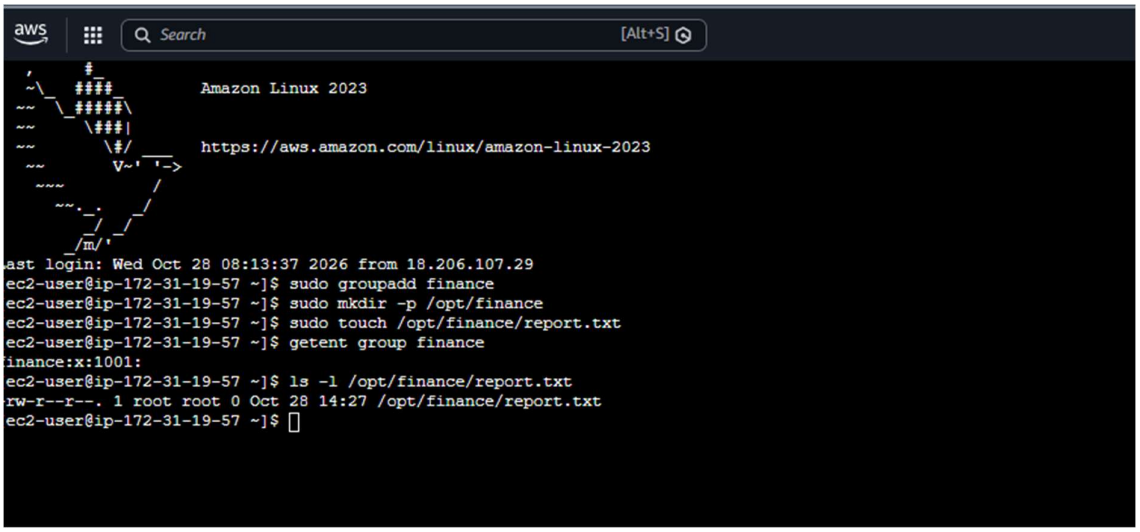
4. d) Extract error entries for a specified date into a separate file.

4.e) Explain how you would calculate the number of errors occurring in each hour.

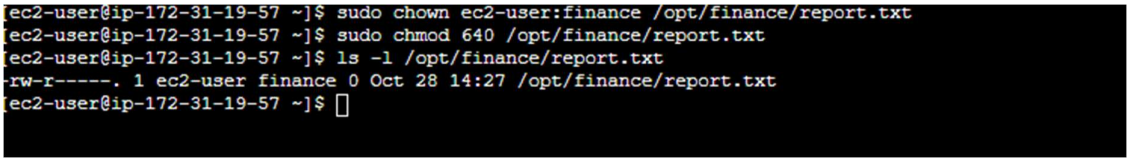


5. Permissions and System Troubleshooting

5. a) . Create a group named finance and a report file, assuming you have the required privileges



5. b)Set permissions so the owner can read and write, the finance group can read, and other users have no access.



5. c) Identify the processes consuming the most CPU.

```
op - 14:34:48 up  7:20,  1 user,  load average: 0.00, 0.00, 0.00
asks: 104 total,   1 running, 103 sleeping,   0 stopped,   0 zombie
Cpu(s):  0.2 us,  0.0 sy,  0.0 ni, 99.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
iB Mem :    913.1 total,    485.2 free,    170.2 used,    257.7 buff/cache
iB Swap:      0.0 total,      0.0 free,      0.0 used.    611.1 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0  173928  17920  11244 S   0.0   1.9   0:02.56 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.09 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.29 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.08 migration/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     21 root      rt   0       0      0      0 S   0.0   0.0   0:00.11 migration/1
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.10 ksoftirqd/1
     24 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     28 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-inet_frag_wq
     29 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     30 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     31 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     32 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kauditd
     35 root      20   0       0      0      0 S   0.0   0.0   0:00.00 khungtaskd
     36 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
     38 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-writeback
     39 root      20   0       0      0      0 S   0.0   0.0   0:00.74 kcompactd0
     40 root      39  19       0      0      0 S   0.0   0.0   0:00.00 khugepaged
     41 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kblockd
     42 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-blkcg_punt_bio
     43 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kintegrityd
     44 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 irq/9-acpi
     45 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-tpm_dev_wq
     46 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-md_bitmap
     47 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-edac-poller
     48 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 watchdogd
     49 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-quota_events_unbound
     51 root       0 -20       0      0      0 I   0.0   0.0   0:00.03 kworker/0:1H-kblockd
     75 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kswapd0
     78 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-xfsalloc
     79 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-xfs_mru_cache
```

5. d) Display memory usage and filesystem disk utilization.

```
[ec2-user@ip-172-31-19-57 ~]$ free -h
               total        used        free      shared  buff/cache   available
Mem:           913Mi       170Mi       485Mi       0.0Ki       257Mi       611Mi
Swap:             0B          0B          0B
[ec2-user@ip-172-31-19-57 ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M     0  4.0M   0% /dev
tmpfs           457M     0  457M   0% /dev/shm
tmpfs           183M  444K  183M   1% /run
efivarfs        128K  2.7K  121K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p1  8.0G  1.6G  6.4G  20% /
tmpfs           457M  4.0K  457M   1% /tmp
/dev/nvme0n1p128  10M  1.3M  8.7M  13% /boot/efi
tmpfs            92M     0   92M   0% /run/user/1000
[ec2-user@ip-172-31-19-57 ~]$
```

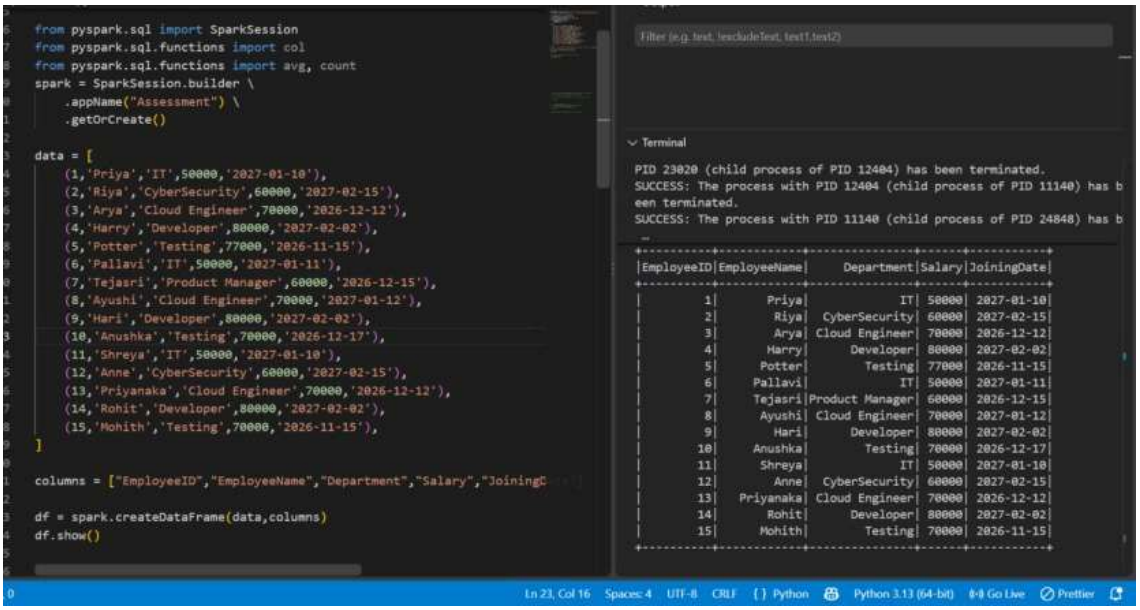
5. e) Find files larger than 100 MB under a specified application directory.

```
tmpfs            92M     0   92M   0% /run/user/1000
[ec2-user@ip-172-31-19-57 ~]$ sudo find /opt/myapp -type f -size +100M -print
find: '/opt/myapp': No such file or directory
[ec2-user@ip-172-31-19-57 ~]$ sudo mkdir -p /opt/myapp
[ec2-user@ip-172-31-19-57 ~]$
```

SECTION C: PYSPARK DATA ENGINEERING

7. PySpark DataFrame Operations

7. a) Display the schema and first five records.



```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
from pyspark.sql.functions import avg, count
spark = SparkSession.builder \
    .appName("Assessment") \
    .getOrCreate()

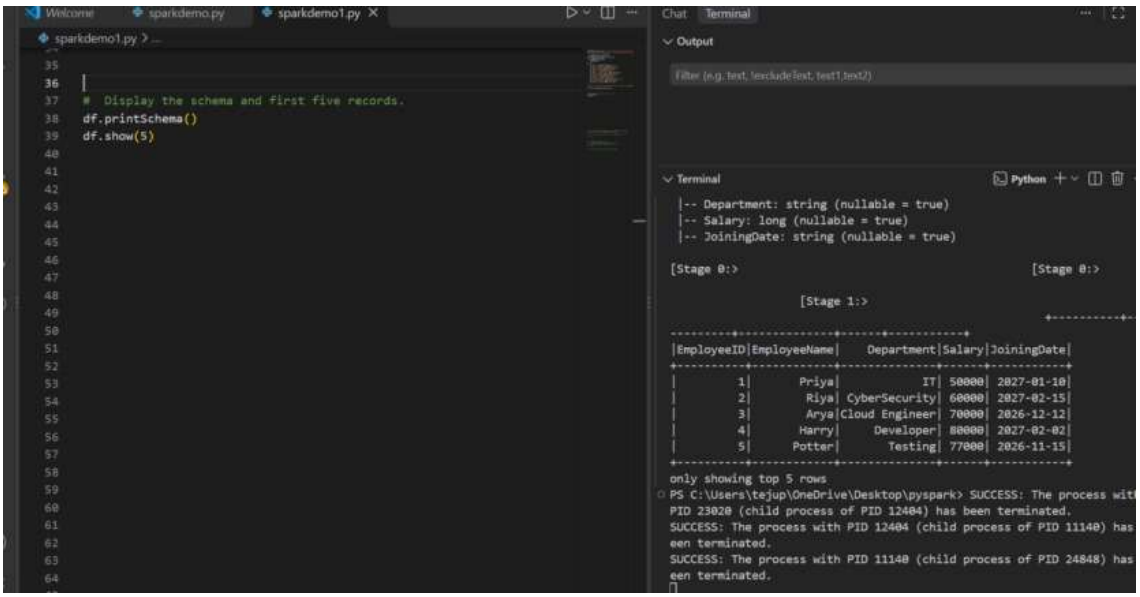
data = [
    (1,'Priya','IT',50000,'2027-01-10'),
    (2,'Riya','CyberSecurity',60000,'2027-02-15'),
    (3,'Arya','Cloud Engineer',70000,'2026-12-12'),
    (4,'Harry','Developer',80000,'2027-02-02'),
    (5,'Potter','Testing',77000,'2026-11-15'),
    (6,'Pallavi','IT',50000,'2027-01-11'),
    (7,'Tejasri','Product Manager',60000,'2026-12-15'),
    (8,'Ayushi','Cloud Engineer',70000,'2027-01-12'),
    (9,'Hari','Developer',80000,'2027-02-02'),
    (10,'Anushka','Testing',70000,'2026-12-17'),
    (11,'Shreya','IT',50000,'2027-01-10'),
    (12,'Anne','CyberSecurity',60000,'2027-02-15'),
    (13,'Priyanaka','Cloud Engineer',70000,'2026-12-12'),
    (14,'Rohit','Developer',80000,'2027-02-02'),
    (15,'Mohith','Testing',70000,'2026-11-15'),
]

columns = ["EmployeeID","EmployeeName","Department","Salary","JoiningD

df = spark.createDataFrame(data,columns)
df.show()
```

```
Terminal
PID 23020 (child process of PID 12404) has been terminated.
SUCCESS: The process with PID 12404 (child process of PID 11140) has b
een terminated.
SUCCESS: The process with PID 11140 (child process of PID 24848) has b
+----------+------------+--------------+------+----------+
|EmployeeID|EmployeeName|    Department|Salary|JoiningDate|
+----------+------------+--------------+------+----------+
|         1|       Priya|            IT| 50000| 2027-01-10|
|         2|        Riya|  CyberSecurity| 60000| 2027-02-15|
|         3|        Arya| Cloud Engineer| 70000| 2026-12-12|
|         4|       Harry|     Developer| 80000| 2027-02-02|
|         5|      Potter|       Testing| 77000| 2026-11-15|
|         6|     Pallavi|            IT| 50000| 2027-01-11|
|         7|     Tejasri|Product Manager| 60000| 2026-12-15|
|         8|      Ayushi| Cloud Engineer| 70000| 2027-01-12|
|         9|        Hari|     Developer| 80000| 2027-02-02|
|        10|     Anushka|       Testing| 70000| 2026-12-17|
|        11|      Shreya|            IT| 50000| 2027-01-10|
|        12|        Anne|  CyberSecurity| 60000| 2027-02-15|
|        13|   Priyanaka| Cloud Engineer| 70000| 2026-12-12|
|        14|       Rohit|     Developer| 80000| 2027-02-02|
|        15|      Mohith|       Testing| 70000| 2026-11-15|
+----------+------------+--------------+------+----------+
```

7. b) Filter employees whose monthly salary is greater than ₹60,000.

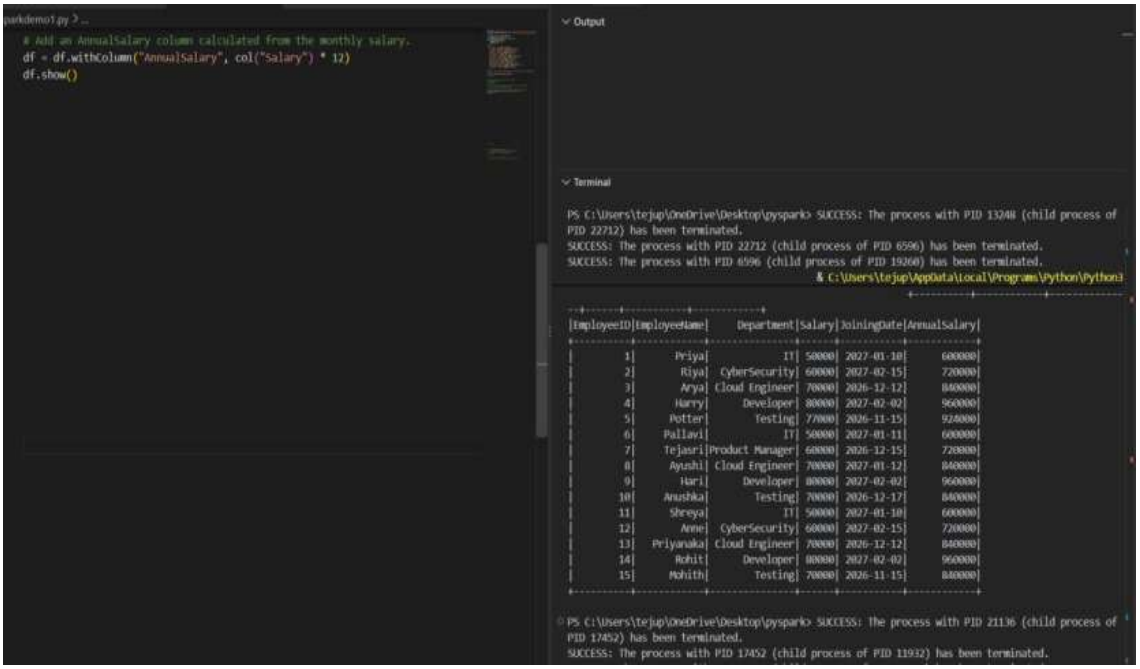


```
# Display the schema and first five records.
df.printSchema()
df.show(5)
```

```
Output
Terminal
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: string (nullable = true)

[Stage 0:>                                                          [Stage 0:>
                [Stage 1:>
+----------+------------+--------------+------+----------+
|EmployeeID|EmployeeName|    Department|Salary|JoiningDate|
+----------+------------+--------------+------+----------+
|         1|       Priya|            IT| 50000| 2027-01-10|
|         2|        Riya|  CyberSecurity| 60000| 2027-02-15|
|         3|        Arya| Cloud Engineer| 70000| 2026-12-12|
|         4|       Harry|     Developer| 80000| 2027-02-02|
|         5|      Potter|       Testing| 77000| 2026-11-15|
+----------+------------+--------------+------+----------+
only showing top 5 rows
PS C:\Users\tejup\OneDrive\Desktop\pyspark> SUCCESS: The process wit
PID 23020 (child process of PID 12404) has been terminated.
SUCCESS: The process with PID 12404 (child process of PID 11140) has
een terminated.
SUCCESS: The process with PID 11140 (child process of PID 24848) has
een terminated.
```

7. c) Add an AnnualSalary column calculated from the monthly salary.



```
# Add an AnnualSalary column calculated from the monthly salary.
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.show()
```

```
Output

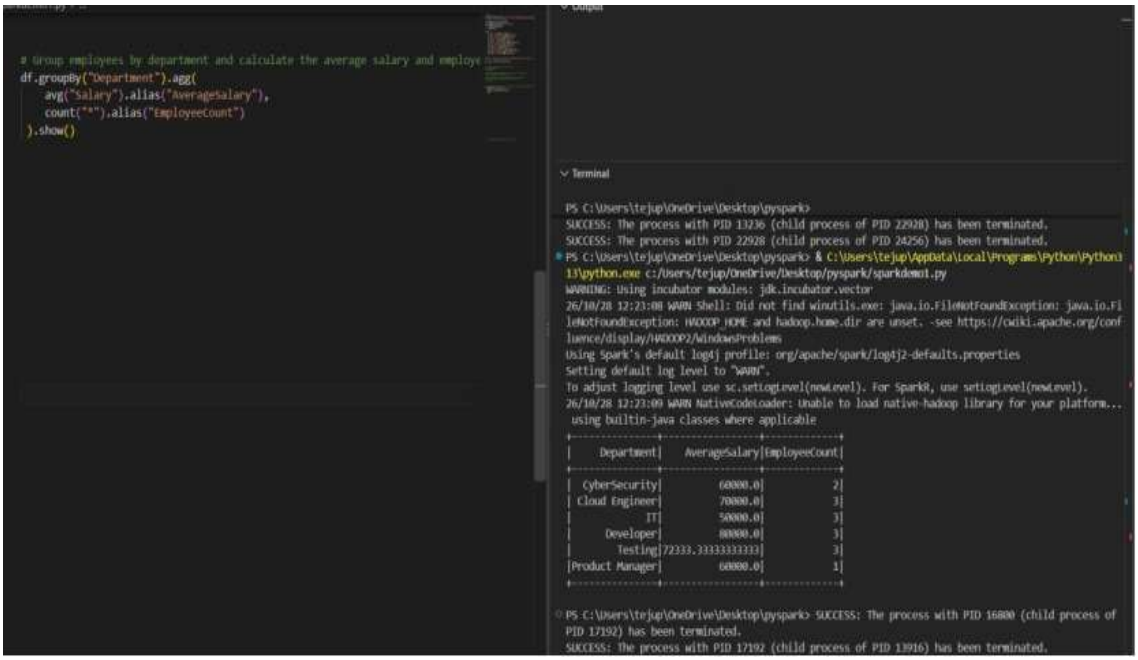
Terminal

PS C:\Users\tejup\OneDrive\Desktop\pyspark> SUCCESS: The process with PID 13248 (child process of
PID 22712) has been terminated.
SUCCESS: The process with PID 22712 (child process of PID 6596) has been terminated.
SUCCESS: The process with PID 6596 (child process of PID 19260) has been terminated.
                                                & C:\Users\tejup\AppData\Local\Programs\Python\Python3

+----------+------------+---------------+------+----------+------------+
|EmployeeID|EmployeeName|     Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+---------------+------+----------+------------+
|         1|       Priya|             IT| 50000| 2027-01-10|      600000|
|         2|        Riya|  CyberSecurity| 60000| 2027-02-15|      720000|
|         3|        Arya| Cloud Engineer| 70000| 2026-12-12|      840000|
|         4|       Harry|      Developer| 80000| 2027-02-02|      960000|
|         5|      Potter|        Testing| 77000| 2026-11-15|      924000|
|         6|     Pallavi|             IT| 50000| 2027-01-11|      600000|
|         7|     Tejasri|Product Manager| 60000| 2026-12-15|      720000|
|         8|      Ayushi| Cloud Engineer| 70000| 2027-01-12|      840000|
|         9|        Hari|      Developer| 80000| 2027-02-02|      960000|
|        10|     Anushka|        Testing| 70000| 2026-12-17|      840000|
|        11|      Shreya|             IT| 50000| 2027-01-10|      600000|
|        12|        Anne|  CyberSecurity| 60000| 2027-02-15|      720000|
|        13|   Priyanaka| Cloud Engineer| 70000| 2026-12-12|      840000|
|        14|       Rohit|      Developer| 80000| 2027-02-02|      960000|
|        15|      Mohith|        Testing| 70000| 2026-11-15|      840000|
+----------+------------+---------------+------+----------+------------+

PS C:\Users\tejup\OneDrive\Desktop\pyspark> SUCCESS: The process with PID 21136 (child process of
PID 17452) has been terminated.
SUCCESS: The process with PID 17452 (child process of PID 11932) has been terminated.
```

7. d) Group employees by department and calculate the average salary and employee count.



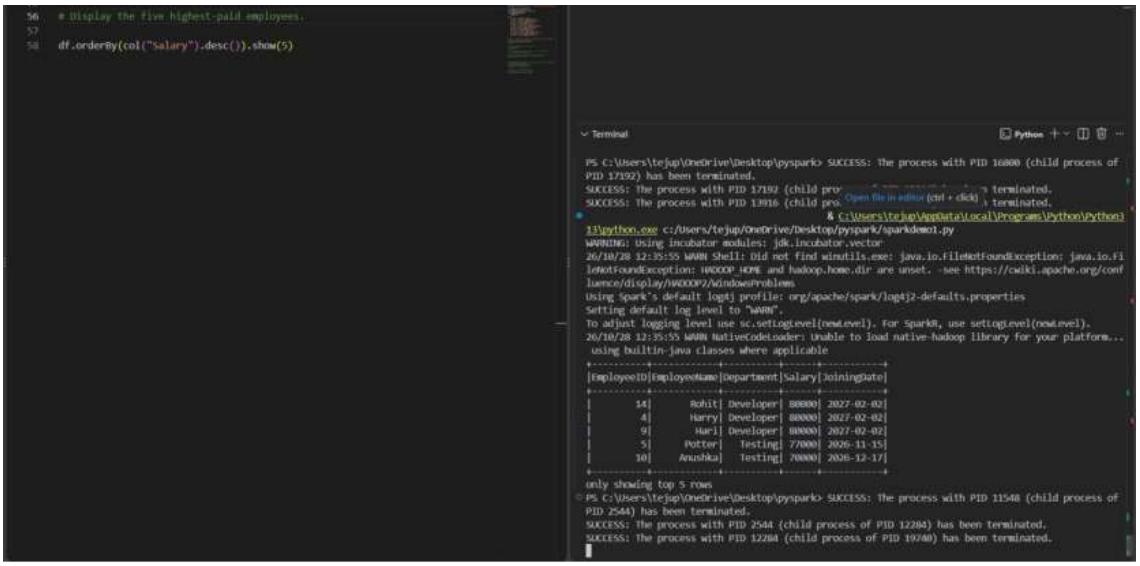
```
# Group employees by department and calculate the average salary and employe
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()
```

```
Terminal

PS C:\Users\tejup\OneDrive\Desktop\pyspark>
SUCCESS: The process with PID 13236 (child process of PID 22928) has been terminated.
SUCCESS: The process with PID 22928 (child process of PID 24256) has been terminated.
PS C:\Users\tejup\OneDrive\Desktop\pyspark> & C:\Users\tejup\AppData\Local\Programs\Python\Python3
13\python.exe c:/Users/tejup/OneDrive/Desktop/pyspark/sparkdemo1.py
WARNING: Using incubator modules: jdk.incubator.vector
26/10/28 12:23:08 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.Fi
leNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see https://cwiki.apache.org/conf
luence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/28 12:23:09 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...
 using builtin-java classes where applicable
+---------------+-----------------+-------------+
|     Department|    AverageSalary|EmployeeCount|
+---------------+-----------------+-------------+
|  CyberSecurity|          60000.0|            2|
| Cloud Engineer|          70000.0|            3|
|             IT|          50000.0|            3|
|      Developer|          80000.0|            3|
|        Testing|72333.33333333333|            3|
|Product Manager|          60000.0|            1|
+---------------+-----------------+-------------+

PS C:\Users\tejup\OneDrive\Desktop\pyspark> SUCCESS: The process with PID 16800 (child process of
PID 17192) has been terminated.
SUCCESS: The process with PID 17192 (child process of PID 13916) has been terminated.
```

7. e) Display the five highest-paid employees.



9. PySpark Joins and Performance

9. a) Load both datasets into DataFrames.

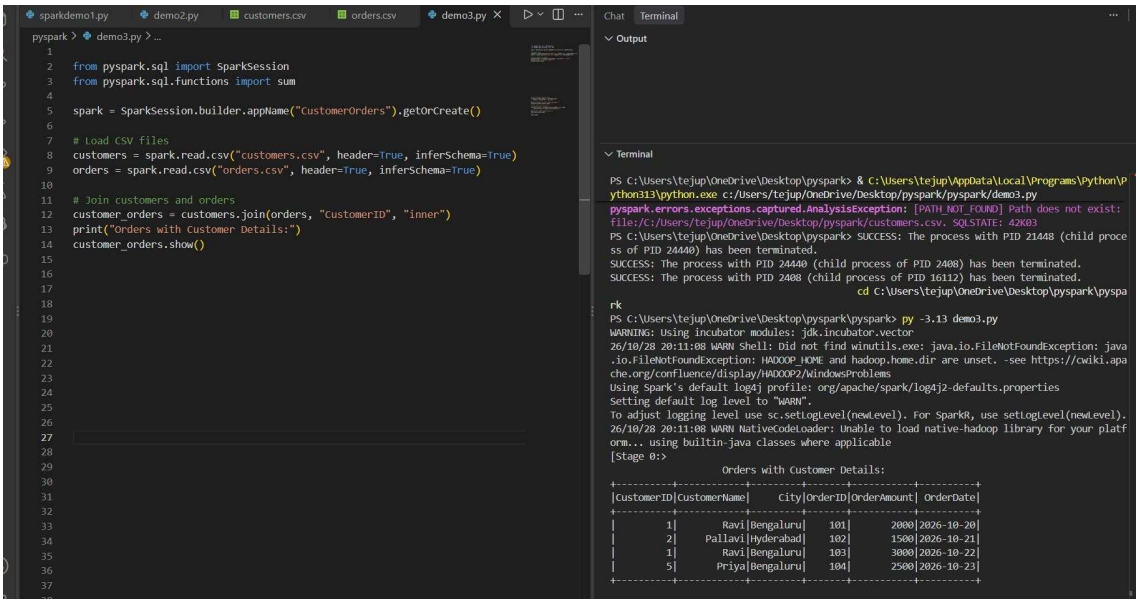
9. b) Join the datasets to display orders with customer details.

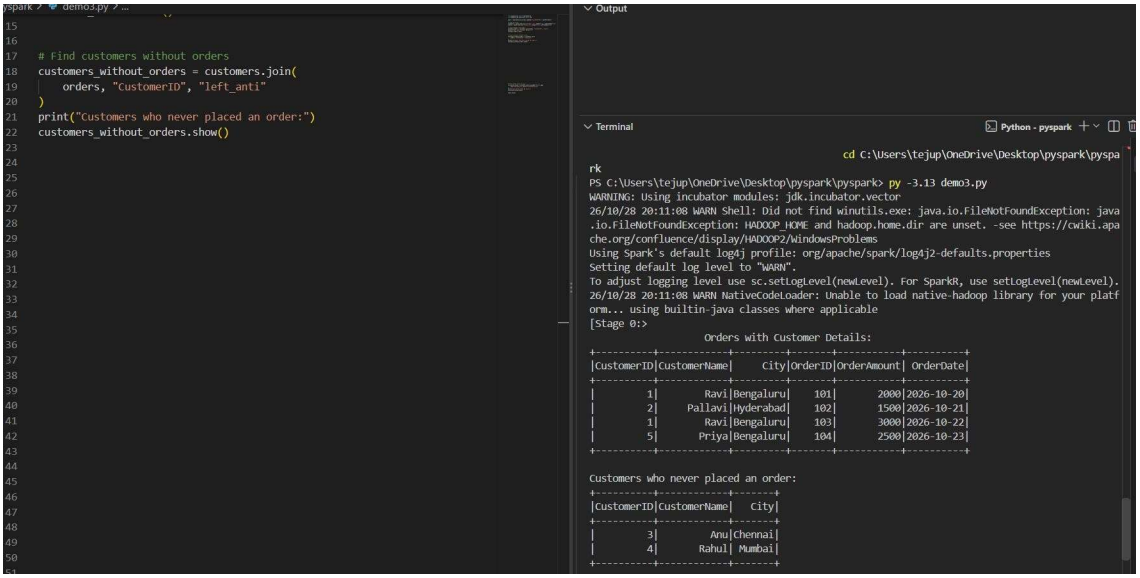
9. c) Identify customers who have never placed an order.

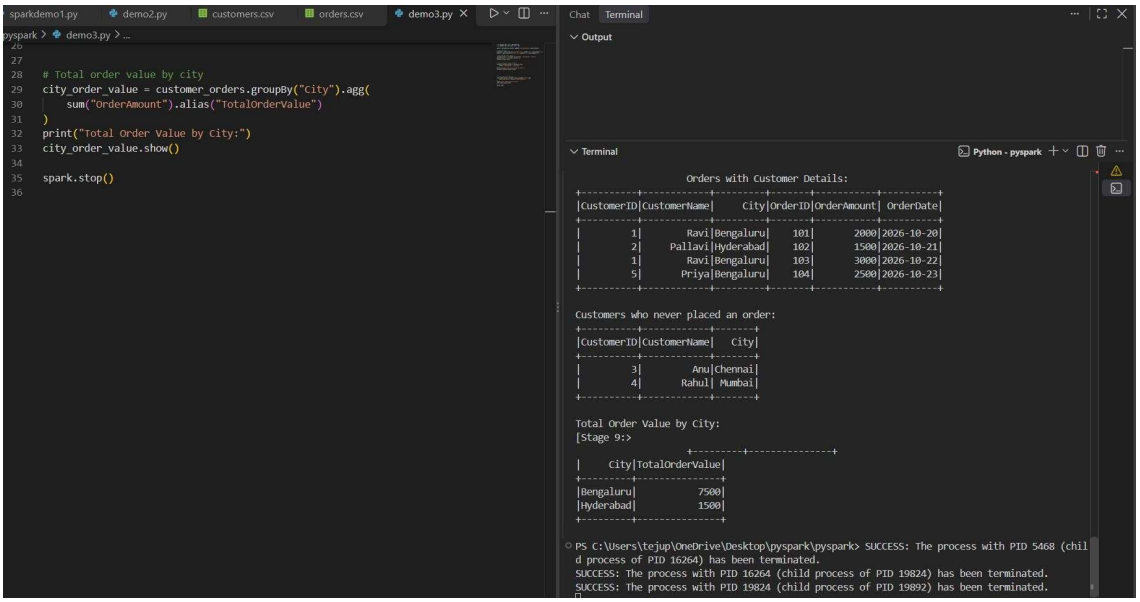
9. d) Calculate the total order value by city.

9. e) Explain when a broadcast join is useful and what conditions make it appropriate.

9. f) Explain the difference between transformations and actions, giving one example of each.


sparkdemo1.py
demo2.py
customers.csv
orders.csv
demo3.py
pyspark > demo3.py > ...
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum
spark = SparkSession.builder.appName("CustomerOrders").getOrCreate()
Load CSV files
customers = spark.read.csv("customers.csv", header=True, inferSchema=True)
orders = spark.read.csv("orders.csv", header=True, inferSchema=True)
Join customers and orders
customer_orders = customers.join(orders, "CustomerID", "inner")
print("Orders with Customer Details:")
customer_orders.show()
Chat
Terminal
Output
Terminal
PS C:\Users\tejup\OneDrive\Desktop\pyspark> & C:\Users\tejup\AppData\Local\Programs\Python\Python313\python.exe c:/Users/tejup/OneDrive/Desktop/pyspark/pyspark/demo3.py
pyspark.errors.exceptions.captured.AnalysisException: [PATH_NOT_FOUND] Path does not exist: file:/C:/Users/tejup/OneDrive/Desktop/pyspark/customers.csv. SQLSTATE: 42K03
PS C:\Users\tejup\OneDrive\Desktop\pyspark> SUCCESS: The process with PID 21448 (child process of PID 24440) has been terminated.
SUCCESS: The process with PID 24440 (child process of PID 2408) has been terminated.
SUCCESS: The process with PID 2408 (child process of PID 16112) has been terminated.
cd C:\Users\tejup\OneDrive\Desktop\pyspark\pyspark
PS C:\Users\tejup\OneDrive\Desktop\pyspark\pyspark> py -3.13 demo3.py
WARNING: Using incubator modules: jdk.incubator.vector
26/10/28 20:11:08 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see https://cwiki.apache.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/28 20:11:08 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[Stage 0:>
Orders with Customer Details:
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName| City|OrderID|OrderAmount| OrderDate|
+----------+------------+---------+-------+-----------+----------+
1	Ravi	Bengaluru	101	2000	2026-10-20
2	Pallavi	Hyderabad	102	1500	2026-10-21
1	Ravi	Bengaluru	103	3000	2026-10-22
5	Priya	Bengaluru	104	2500	2026-10-23
+----------+------------+---------+-------+-----------+----------+

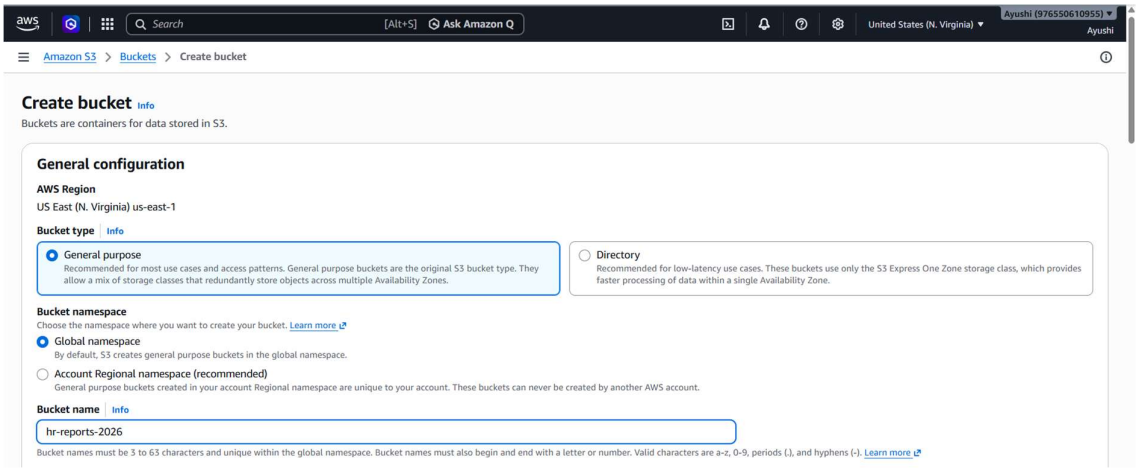

Find customers without orders
customers_without_orders = customers.join(
orders, "CustomerID", "left_anti"
)
print("Customers who never placed an order:")
customers_without_orders.show()
Output
Terminal
Python - pyspark
cd C:\Users\tejup\OneDrive\Desktop\pyspark\pyspark
PS C:\Users\tejup\OneDrive\Desktop\pyspark\pyspark> py -3.13 demo3.py
WARNING: Using incubator modules: jdk.incubator.vector
26/10/28 20:11:08 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see https://cwiki.apache.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/28 20:11:08 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[Stage 0:>
Orders with Customer Details:
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName| City|OrderID|OrderAmount| OrderDate|
+----------+------------+---------+-------+-----------+----------+
1	Ravi	Bengaluru	101	2000	2026-10-20
2	Pallavi	Hyderabad	102	1500	2026-10-21
1	Ravi	Bengaluru	103	3000	2026-10-22
5	Priya	Bengaluru	104	2500	2026-10-23
+----------+------------+---------+-------+-----------+----------+
Customers who never placed an order:
+----------+------------+-------+
|CustomerID|CustomerName| City|
+----------+------------+-------+
| 3| Anu|Chennai|
| 4| Rahul| Mumbai|
+----------+------------+-------+

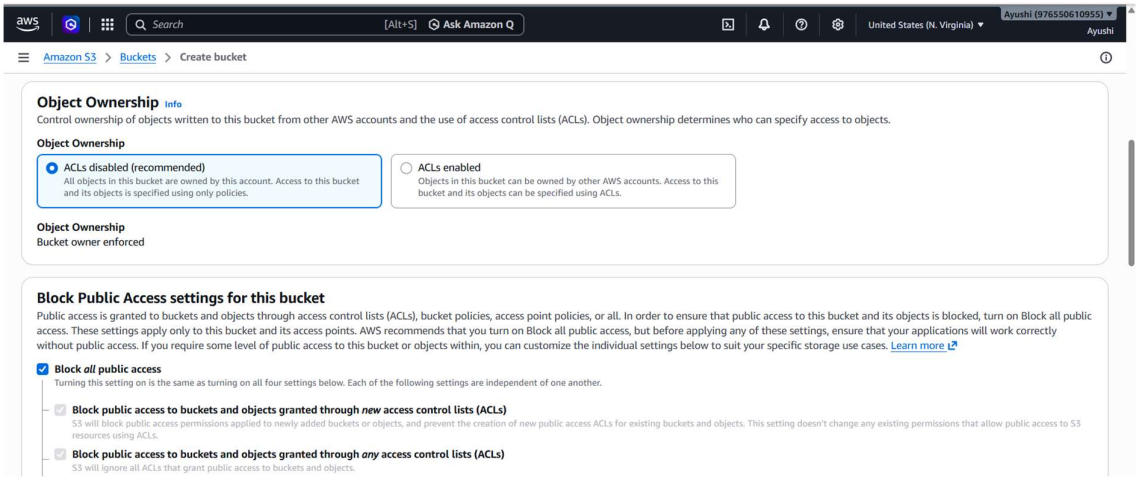

sparkdemo1.py
demo2.py
customers.csv
orders.csv
demo3.py
pyspark > demo3.py > ...
Total order value by city
city_order_value = customer_orders.groupBy("City").agg(
sum("OrderAmount").alias("TotalOrderValue")
)
print("Total Order Value by City:")
city_order_value.show()
spark.stop()
Chat
Terminal
Output
Terminal
Python - pyspark
Orders with Customer Details:
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName| City|OrderID|OrderAmount| OrderDate|
+----------+------------+---------+-------+-----------+----------+
1	Ravi	Bengaluru	101	2000	2026-10-20
2	Pallavi	Hyderabad	102	1500	2026-10-21
1	Ravi	Bengaluru	103	3000	2026-10-22
5	Priya	Bengaluru	104	2500	2026-10-23
+----------+------------+---------+-------+-----------+----------+
Customers who never placed an order:
+----------+------------+-------+
|CustomerID|CustomerName| City|
+----------+------------+-------+
| 3| Anu|Chennai|
| 4| Rahul| Mumbai|
+----------+------------+-------+
Total Order Value by City:
[Stage 9:>
+---------+--------------+
| City|TotalOrderValue|
+---------+--------------+
|Bengaluru| 7500|
|Hyderabad| 1500|
+---------+--------------+
PS C:\Users\tejup\OneDrive\Desktop\pyspark\pyspark> SUCCESS: The process with PID 5468 (child process of PID 16264) has been terminated.
SUCCESS: The process with PID 16264 (child process of PID 19824) has been terminated.
SUCCESS: The process with PID 19824 (child process of PID 19892) has been terminated.

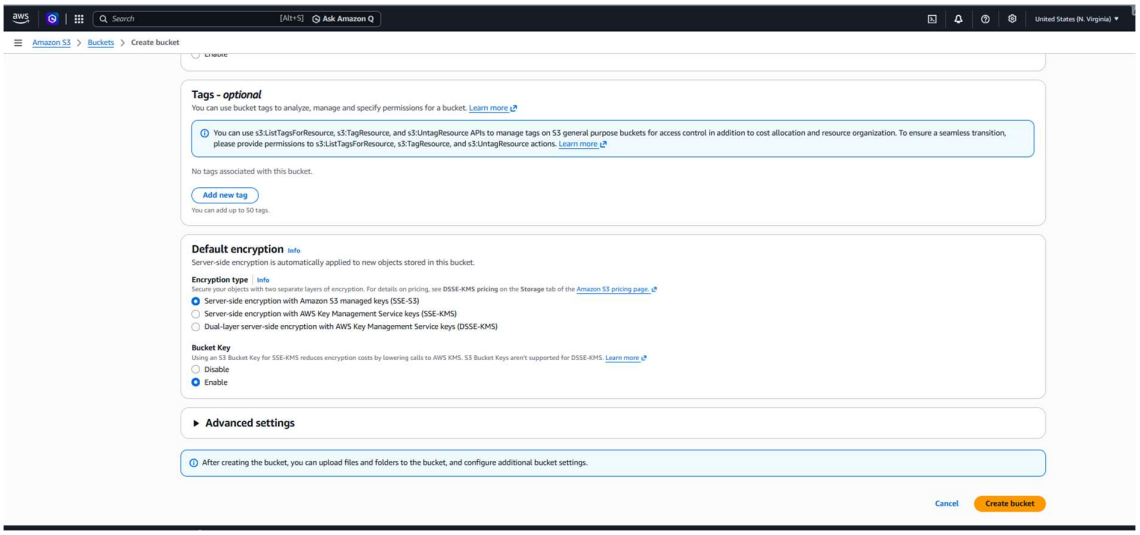
SECTION D: AWS CLOUD SERVICES

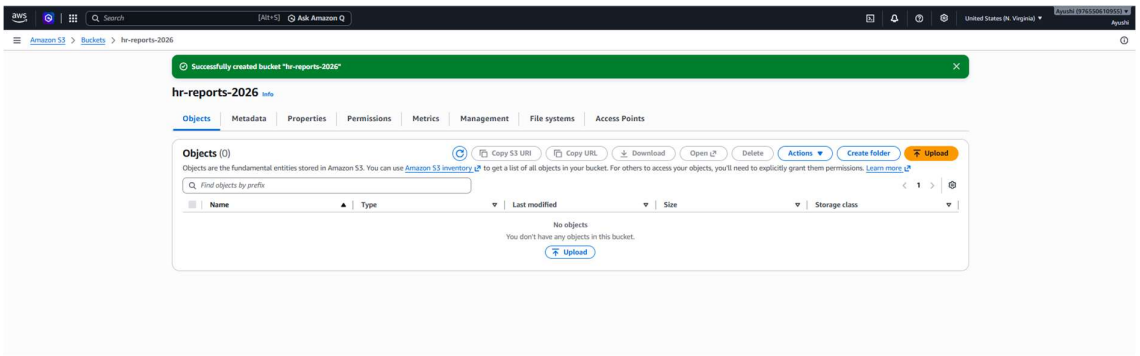
10. S3 and IAM Security

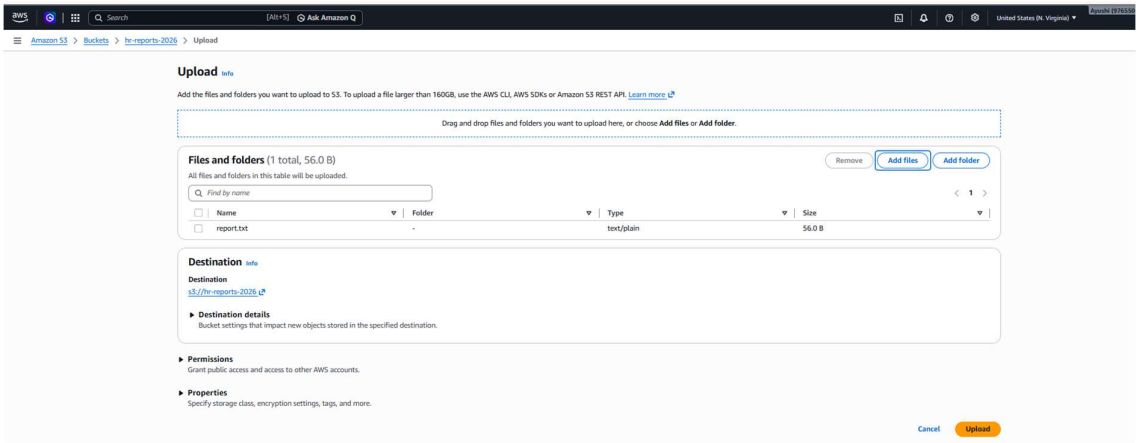
10. a) Create an S3 bucket with Block Public Access enabled and upload a sample file.

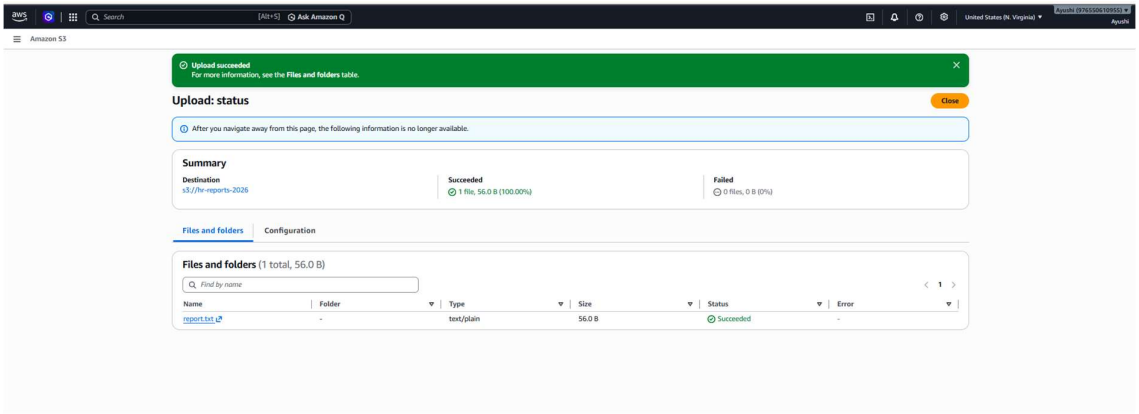




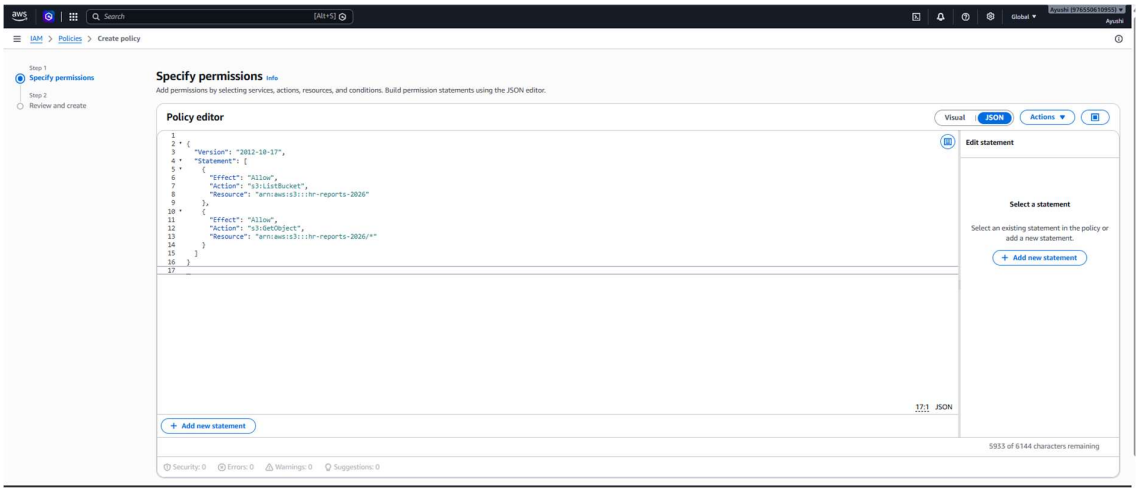

Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
Amazon S3 > Buckets > Create bucket
Tags - optional
You can use bucket tags to analyze, manage and specify permissions for a bucket. Learn more
You can use s3:ListTagsForResource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation and resource organization. To ensure a seamless transition, please provide permissions to s3:ListTagsForResource, s3:TagResource, and s3:UntagResource actions. Learn more
No tags associated with this bucket.
Add new tag
You can add up to 50 tags.
Default encryption Info
Server-side encryption is automatically applied to new objects stored in this bucket.
Encryption type Info
Secure your objects with two separate layers of encryption. For details on pricing, see DSSE-KMS pricing on the Storage tab of the Amazon S3 pricing page.
Server-side encryption with Amazon S3 managed keys (SSE-S3)
Server-side encryption with AWS Key Management Service keys (SSE-KMS)
Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)
Bucket Key
Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. Learn more
Disable
Enable
Advanced settings
After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.
Cancel
Create bucket

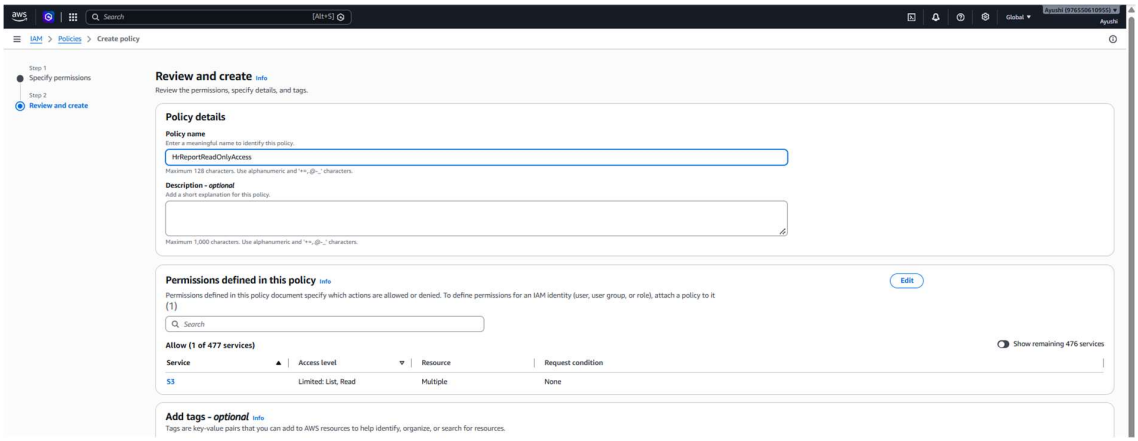

Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
Amazon S3 > Buckets > hr-reports-2026
Successfully created bucket "hr-reports-2026"
hr-reports-2026 Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems
Access Points
Objects (0)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
No objects
You don't have any objects in this bucket.
Upload

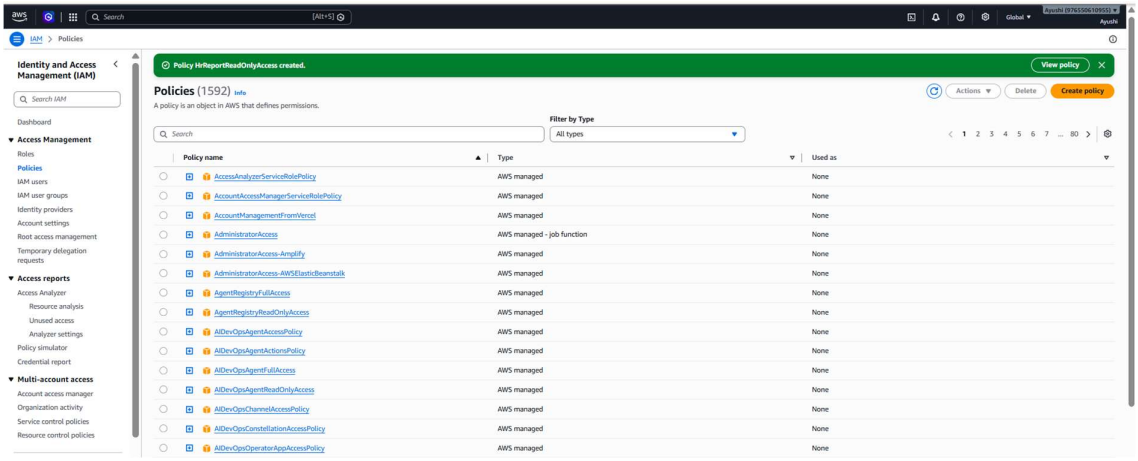

Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
Amazon S3 > Buckets > hr-reports-2026 > Upload
Upload Info
Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDKs or Amazon S3 REST API. Learn more
Drag and drop files and folders you want to upload here, or choose Add files or Add folder.
Files and folders (1 total, 56.0 B)
All files and folders in this table will be uploaded.
Remove
Add files
Add folder
Find by name
Name
Folder
Type
Size
report.txt
-
text/plain
56.0 B
Destination Info
Destination
s3://hr-reports-2026
Destination details
Bucket settings that impact new objects stored in the specified destination.
Permissions
Grant public access and access to other AWS accounts.
Properties
Specify storage class, encryption settings, tags, and more.
Cancel
Upload

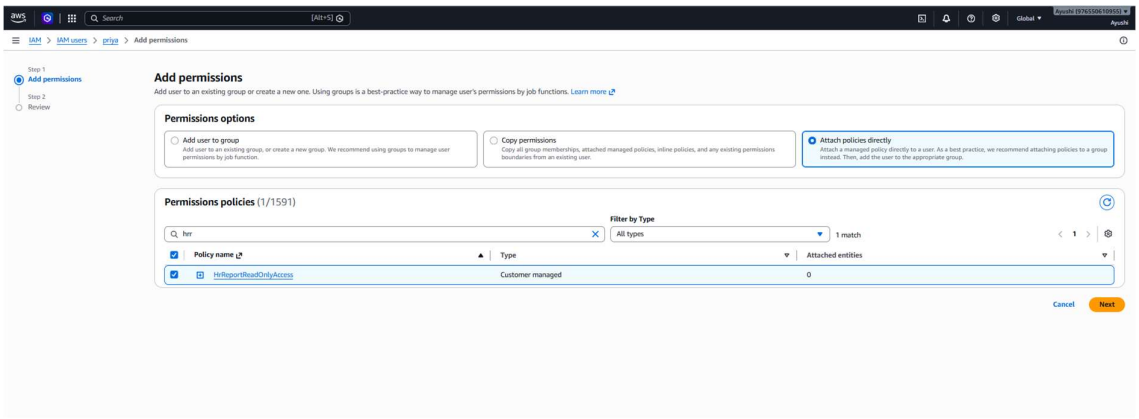


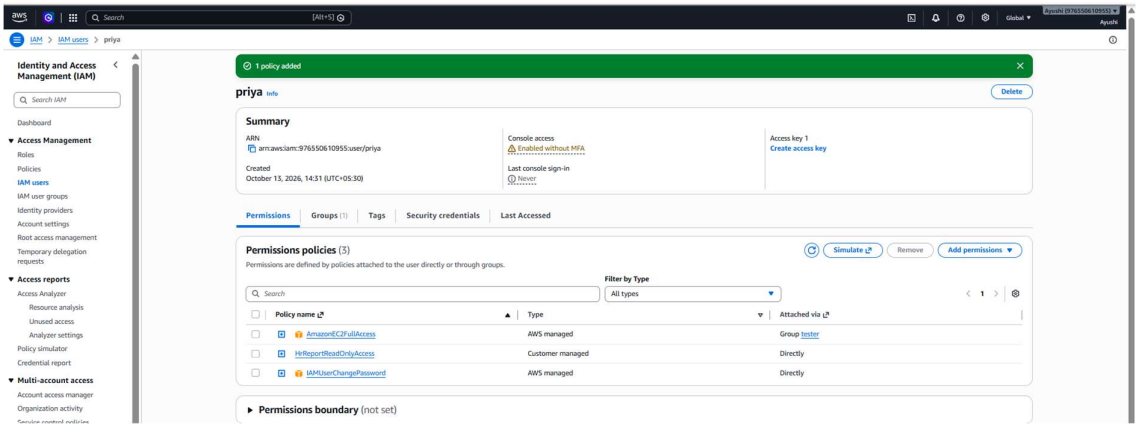
10. b) Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets






Policy HrReportReadOnlyAccess created.
View policy
Policies (1592) Info
A policy is an object in AWS that defines permissions.
Actions
Delete
Create policy
Filter by Type
All types
Policy name
Type
Used as
AccessAnalyzerServiceRolePolicy
AWS managed
None
AccountAccessManagerServiceRolePolicy
AWS managed
None
AccountManagementFromVercel
AWS managed
None
AdministratorAccess
AWS managed - job function
None
AdministratorAccess-Amplify
AWS managed
None
AdministratorAccess-AWSElasticBeanstalk
AWS managed
None
AgentRegistryFullAccess
AWS managed
None
AgentRegistryReadOnlyAccess
AWS managed
None
AIDevOpsAgentAccessPolicy
AWS managed
None
AIDevOpsAgentActionsPolicy
AWS managed
None
AIDevOpsAgentFullAccess
AWS managed
None
AIDevOpsAgentReadOnlyAccess
AWS managed
None
AIDevOpsChannelAccessPolicy
AWS managed
None
AIDevOpsConstellationAccessPolicy
AWS managed
None
AIDevOpsOperatorAppAccessPolicy
AWS managed
None


Add permissions
Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. Learn more
Permissions options
Add user to group
Copy permissions
Attach policies directly
Permissions policies (1/1591)
Filter by Type
hrr
All types
1 match
Policy name
Type
Attached entities
HrReportReadOnlyAccess
Customer managed
0
Cancel
Next


1 policy added
priya Info
Delete
Summary
ARN
arn:aws:iam::976550610955:user/priya
Console access
Enabled without MFA
Access key 1
Create access key
Created
October 13, 2026, 14:31 (UTC+05:30)
Last console sign-in
Never
Permissions
Groups (1)
Tags
Security credentials
Last Accessed
Permissions policies (3)
Permissions are defined by policies attached to the user directly or through groups.
Simulate
Remove
Add permissions
Filter by Type
All types
Policy name
Type
Attached via
AmazonEC2FullAccess
AWS managed
Group tester
HrReportReadOnlyAccess
Customer managed
Directly
IAMUserChangePassword
AWS managed
Directly
Permissions boundary (not set)

10. c) Explain how you would verify that an unauthorized user cannot read the file.

- To verify that an unauthorized user cannot read a file in AWS, I would configure IAM policies to allow access only to authorized users. Then, I would attempt to access the file using an unauthorized user account. If AWS returns an AccessDenied error, it indicates that the user does not have permission to read the file. I would also review IAM policies, S3 bucket policies, and AWS CloudTrail logs to confirm that access is properly restricted and monitored.

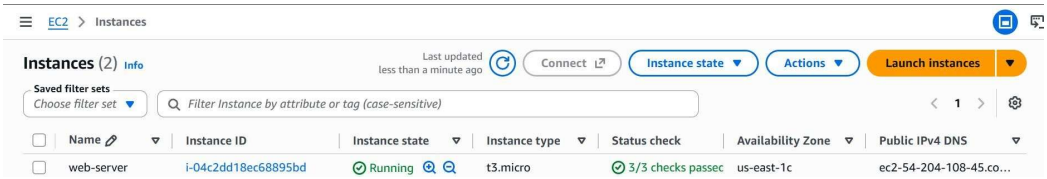
10. d) Explain how encryption at rest protects the stored data.

- Encryption at rest protects stored data by converting it into unreadable ciphertext using encryption algorithms. In AWS, services such as Amazon S3 and Amazon EBS support encryption at rest, and AWS Key Management Service (AWS KMS) can manage the encryption keys. Only authorized users and applications with the required permissions can decrypt and access the original data. Thus, encryption at rest adds an extra layer of security and helps prevent unauthorized users from reading stored data.

11. EC2 Troubleshooting and CloudWatch

11. a) Check the EC2 instance state and status checks.

- In EC2 console, select the instance and check Instance state (running/stopped) and Status checks (system and instance checks). A failed check may indicate host or guest OS/network problems.

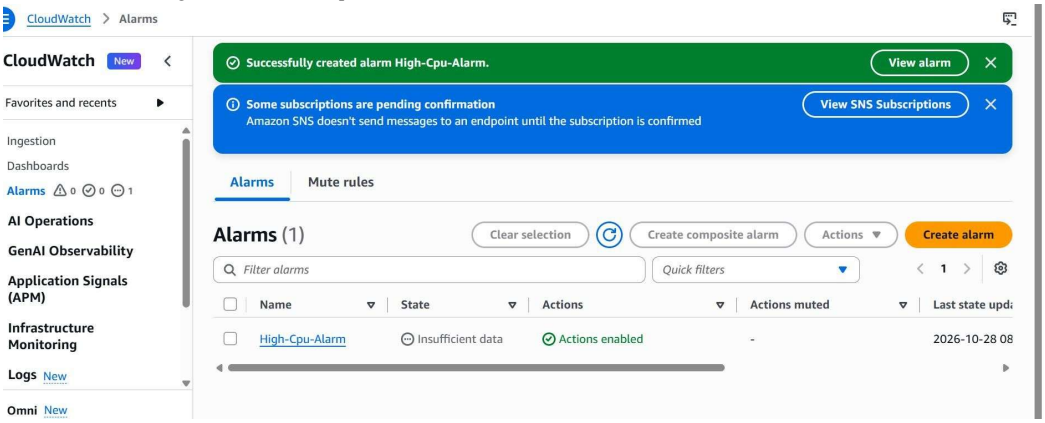


11. b) Explain how security groups and route tables can affect application connectivity.

- Security groups are stateful virtual firewalls; inbound rules must allow the application port from the intended client/source, and outbound rules must permit required traffic. Route tables determine paths between subnets, internet gateways, NAT gateways, and other networks. A public subnet alone does not guarantee reachability; a public IP and suitable routes/firewall rules may also be needed.

11. c) Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

- Create a CloudWatch alarm for the EC2 metric CPU Utilization. Select the instance, set a threshold greater than 85%, choose the statistic (often Average), period (for example 5 minutes), evaluation periods (for example 2 consecutive periods), and configure an SNS notification if required. The alarm triggers when the selected datapoints meet the threshold rule. Tune period/evaluation count to avoid reacting to brief harmless spikes.



11. d) Identify one method for investigating the root cause of high CPU utilization.

- Investigate using CloudWatch CPU and related metrics, EC2 system logs/console output, application logs, and top/ps inside the instance. Identify the process consuming CPU and correlate the spike with deployments, traffic, scheduled jobs, or errors.

Question 12: Event-Driven Processing

Architecture: Amazon S3 input bucket/prefix → S3 Object Created event notification → AWS Lambda function → validate and process CSV → write results to a separate S3 output bucket/prefix. For longer or multi-step jobs, Lambda can start AWS Step Functions or submit an AWS Glue job instead.

12. a) Identify a suitable service for executing the processing code.

- Use AWS Lambda for short, event-driven processing. For large-scale Spark ETL, use AWS Glue; for multi-step orchestration, use Step Functions.

12. b) Explain how an S3 upload can trigger processing.

- Configure an S3 Object Created notification filtered to the input prefix and .csv suffix to invoke Lambda. Ensure Lambda has permission for S3 to invoke it.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": ["s3:GetObject"],
            "Resource": "arn:aws:s3:::source-csv-bucket/*"
        },
        {
            "Effect": "Allow",
            "Action": ["s3:PutObject"],
            "Resource": "arn:aws:s3:::destination-processed-bucket/*"
        },
        {
            "Effect": "Allow",
            "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "lo
            "Resource": "arn:aws:logs:*:*:*"
        }
    ]
}
```

12. c) Explain the IAM permissions needed to read the input file and write the results.

- The execution role should have s3:GetObject for the input prefix and s3:PutObject for the output prefix only, plus necessary logging permissions (such as CloudWatch Logs). Add KMS permissions if SSE-KMS is used.

12. d) Explain how errors and processing logs can be captured.

- Write logs and errors to CloudWatch Logs, configure Lambda error metrics/alarms, and optionally send failure notifications or route asynchronous failures to a dead-letter queue/destination.

12. e) Explain how to prevent duplicate processing or recursive triggering.

- Make processing idempotent by recording processed object keys/version IDs or using a durable idempotency key. Avoid recursive triggers by writing output to a different bucket or a prefix excluded from the trigger filter. S3 events can be delivered more than once, so duplicates must be expected and handled.

SECTION E: AWS STEP FUNCTIONS

13. State Machine Design

Workflow: Start → Validate File → Choice (valid?) → Transform Data → Calculate Sales By Product → StoreResultsInS3 → Send Notification → Success. If validation fails, route to a Fail state or a rejection/notification branch.

13. a) Identify suitable states for validation, processing, storage, and notification.

- The workflow uses AWS Step Functions to coordinate the processing of sales files.
- **ValidateFile:** Checks whether the input file is valid and contains the required data.
- **Choice:** Determines whether the file passes validation.
- **TransformData:** Cleans and transforms the sales data.
- **CalculateSalesByProduct:** Calculates total sales for each product.
- **StoreResultsInS3:** Stores the processed results in Amazon S3.
- **SendNotification:** Sends a notification after successful completion.
- **Fail State:** Stops the workflow if validation fails or an unrecoverable error occurs.

13. b) Explain how a Choice state can route invalid files to a failure path and valid files to processing.

- The Choice state checks the validation result. If the file is valid, the workflow continues to data transformation. If the file is invalid, the workflow follows the failure path and stops processing.

13. c) Explain how Retry and Catch can handle transient failures and permanent errors.

- **Retry** automatically attempts a failed task again when a temporary error occurs, such as service throttling or a temporary network issue. It can use increasing delays between attempts.
- **Catch** handles errors that remain after retries or cannot be recovered. It redirects the workflow to an error-handling state or a failure state.

13. d) Explain how the state machine can invoke Lambda functions or other supported AWS services.

- AWS Step Functions uses Task states to invoke AWS Lambda functions or supported AWS service integrations. Each task performs a particular operation, such as validating a file, transforming data, or storing results. Step Functions manages the order of execution.

13. e) Explain how execution history helps troubleshoot a failed workflow.

- Execution history records the workflow's state transitions, execution status, inputs and outputs, timestamps, and error details. Administrators can identify the state that failed and investigate the cause.

14. Step Functions Implementation

```
{
  "Comment": "Sequential Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:TransformData",
      "Retry": [
        {
          "ErrorEquals": ["States.TaskFailed", "States.Timeout"],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2.0
        }
      ],
      "Next": "LoadResults"
    },
    "LoadResults": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:LoadResults",
      "Catch": [
        {
          "ErrorEquals": ["States.ALL"],
          "Next": "HandleFailure"
        }
      ],
      "End": true
    },
    "HandleFailure": {
      "Type": "Fail",
      "Error": "LoadError",
      "Cause": "The workflow failed during the LoadResults stage."
    }
  }
}
```

14. a) Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

- The workflow contains three tasks that execute in sequence:

1. ValidateFile: Checks the input file. If validation fails, the workflow stops.
2. TransformData: Transforms the validated data. Temporary failures are handled through retries.
3. LoadResults: Loads the processed results into the destination. If loading fails permanently, the error is recorded.

14. b) Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.

- The transformation task uses a retry policy for temporary service errors, such as Lambda service exceptions and throttling.

An appropriate policy can include:

- Initial retry interval: 2 seconds.
- Backoff rate: 2, which increases the delay between attempts.
- Maximum attempts: 3 retries.

This helps recover from temporary failures without immediately stopping the workflow.

```
"Comment": "Sequential Data Processing Workflow",
"StartAt": "ValidateFile",
"States": {
  "ValidateFile": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:ValidateFile",
    "Next": "TransformData"
  },
  "TransformData": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:TransformData",
    "Retry": [
      {
        "ErrorEquals": ["States.TaskFailed", "States.Timeout"],
        "IntervalSeconds": 2,
        "MaxAttempts": 3,
        "BackoffRate": 2.0
      }
    ],
    "Next": "LoadResults"
  },
  "LoadResults": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:LoadResults",
    "Catch": [
      {
        "ErrorEquals": ["States.ALL"],
        "Next": "HandleFailure"
      }
```

14. c) Configure a Catch transition for the loading task that routes to an error-handling state.

- The Catch mechanism captures an error from the loading task and redirects execution to an error-handling task. That task records the failure details, after which the workflow ends in a failed state.

```
    },
    "HandleFailure": {
      "Type": "Fail",
      "Error": "LoadError",
      "Cause": "The workflow failed during the LoadResults stage."
    }
  }
```

14. d) Explain how data from one task is passed to the next task.

- Each task receives input from the workflow and produces an output. By default, the task output becomes the next state's input. This allows the validation result to pass to transformation and the transformed data to pass to the loading task.

14. e) Explain how you would test successful and failed executions.

- **Successful test:** Provide a valid input file and verify that validation, transformation, and loading complete successfully.
- **Failed test:** Provide an invalid file to test validation failure, simulate a temporary transformation error to test retries, and simulate a loading failure to verify error recording.

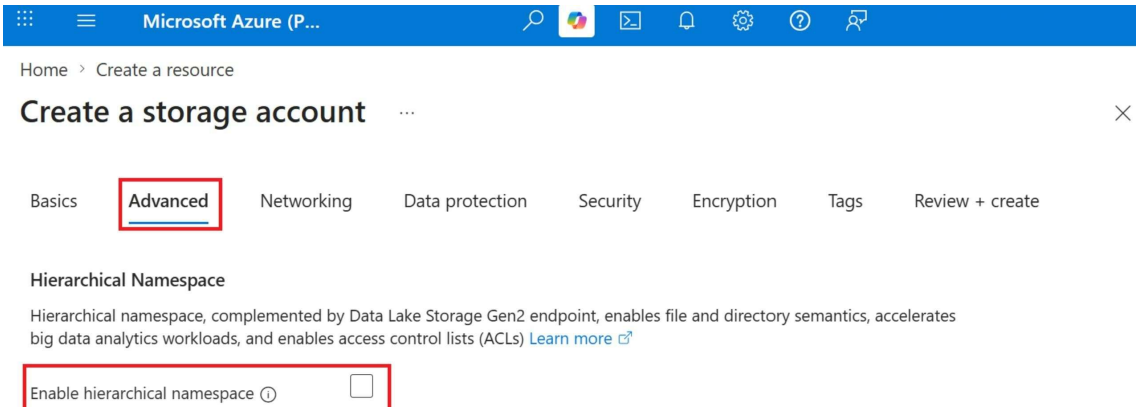
Execution history and Amazon CloudWatch Logs can help verify the results.

SECTION F: MICROSOFT AZURE

15. Azure Data Lake Storage and Security

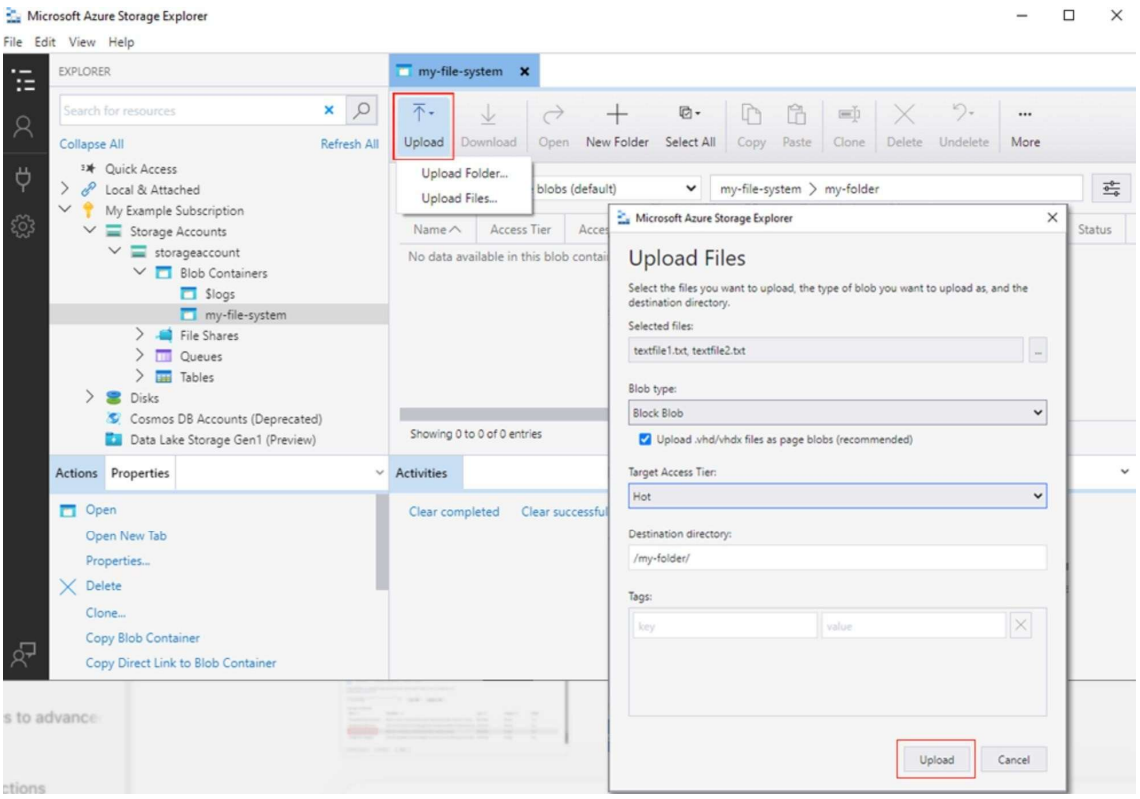
15. a) Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2.

➢ Create a Storage Account in the Azure portal. During configuration, enable the Hierarchical Namespace option to support Azure Data Lake Storage Gen2. Choose the appropriate region, redundancy, and security settings.



15. b) Create a filesystem or container and upload a sample dataset.

➢ Open the storage account and create a container or filesystem, such as raw-data. Upload a sample CSV dataset. Files can be organized into directories for raw and processed data.



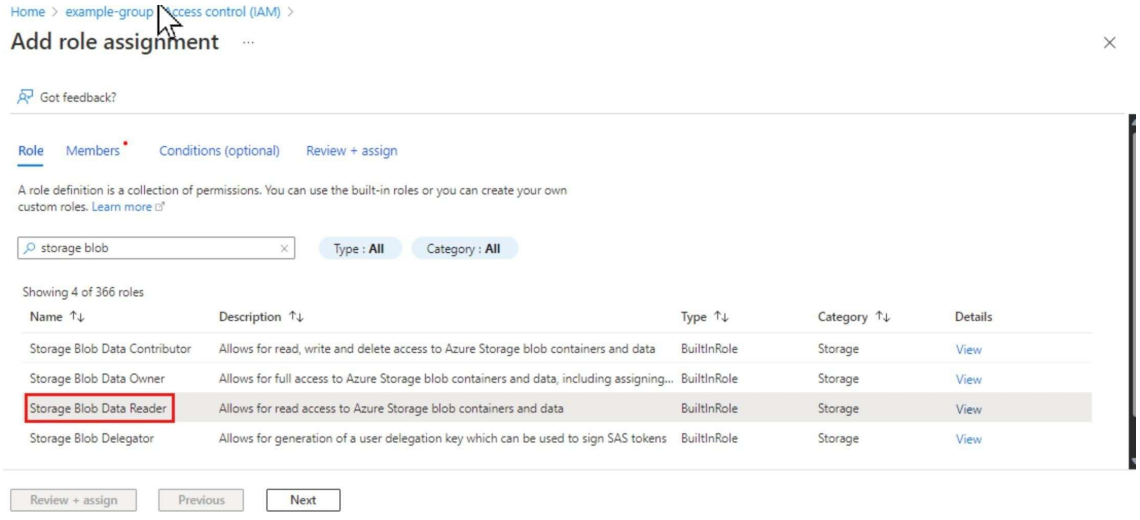
15. c) Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data.

- Microsoft Entra ID provides identity management and authentication. Azure Role-Based Access Control (RBAC) assigns permissions to users, groups, and applications according to their responsibilities.

For example:

- Storage Blob Data Reader: Allows users to read data.
- Storage Blob Data Contributor: Allows users to read, write, and manage data.

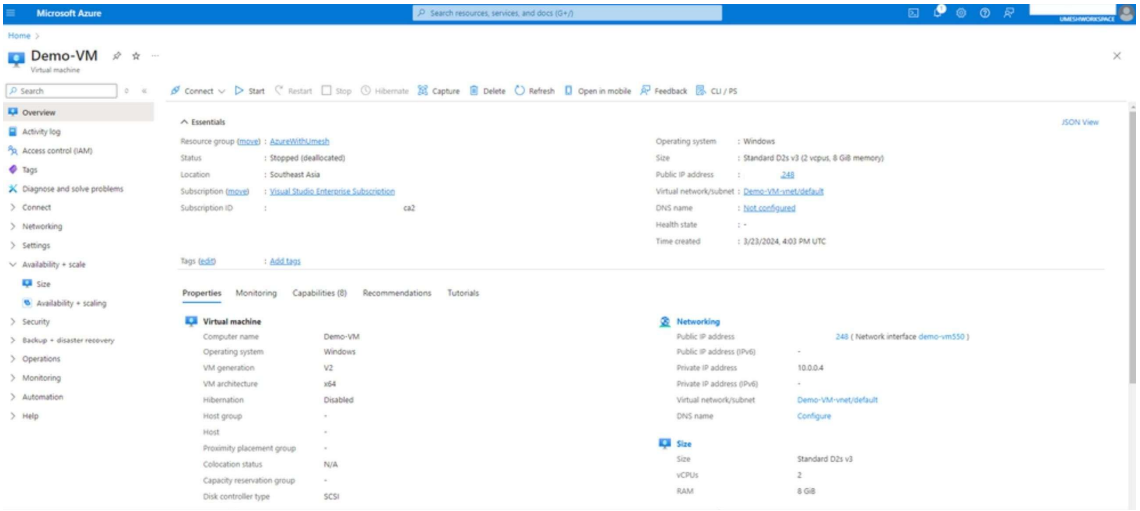
Permissions should be assigned only at the scope required. Access Control Lists can provide additional directory- and file-level permissions in ADLS Gen2.



15. d) Explain how you would verify that unauthorized users cannot access the dataset.

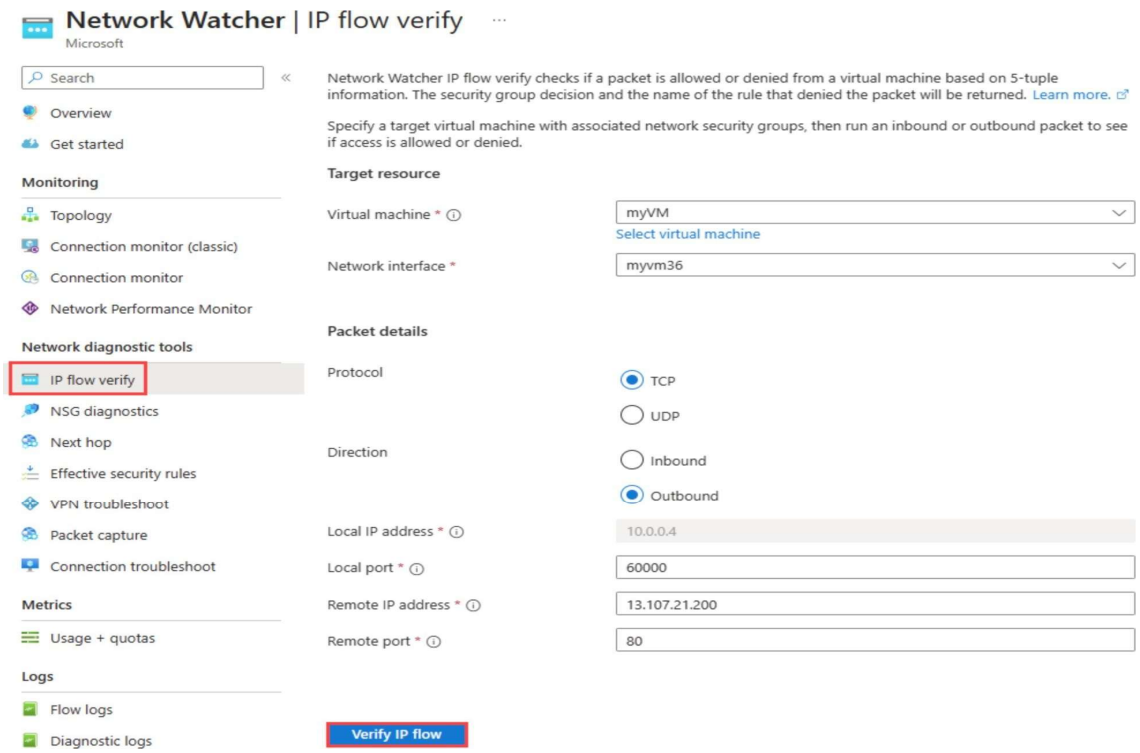
- Test access using a separate account that has not been granted the required permissions. Confirm that access is denied. Also review role assignments, access control lists, network restrictions, and relevant diagnostic logs.

16. Azure VM Troubleshooting



16. a) Verify the VM's provisioning and power states.

- Open the virtual machine in the Azure portal and check its Overview page. Confirm that the provisioning state is Succeeded and the power state is Running. If the VM is unreachable, inspect Boot Diagnostics and Resource Health



16. b) Explain how Network Security Group rules can prevent connectivity.

- Network Security Groups (NSGs) control inbound and outbound network traffic. Incorrect rules may block SSH connections on port 22, RDP connections on port 3389, or the application's required port. Check both subnet-level and network-interface-level NSGs, including rule priorities and permitted source addresses.

16. c) Explain how subnet and routing configuration affect connectivity.

- The VM must be connected to the correct subnet, and its route tables must direct traffic through the appropriate network path. Incorrect routes, unavailable next hops, DNS problems, or missing VPN connectivity can prevent administrators from reaching the VM.

16. d) Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

- Use **Azure Bastion** to connect to the VM through browser-based SSH or RDP without requiring a public IP address on the VM. Private connectivity through a VPN or ExpressRoute is another option.

Administrators should also:

- Check Boot Diagnostics and Serial Console when appropriate.
- Restrict access to trusted source IP addresses.
- Avoid exposing SSH or RDP directly to the entire internet.
- Use Just-in-Time VM access where available

Question 17: Azure Data Integration

17. a) Select an appropriate service to orchestrate data movement.

- Use Azure Data Factory (ADF) to create and manage a pipeline that orchestrates the movement and transformation of customer transaction data.

17. b) Explain how to copy the CSV file into Azure Data Lake Storage Gen2.

- Create linked services in Azure Data Factory for the source location and the ADLS Gen2 destination. Configure a Copy activity to read the CSV file and transfer it to the appropriate storage directory.

17. c) Describe how to organize files by ingestion date.

- Store files in date-based directories, using a structure such as year, month, and day. This makes the data easier to locate, manage, and process for a particular date.

17. d) Explain how to handle malformed records and missing values.

- Validate the CSV schema and check for missing or incorrectly formatted values. Depending on business requirements, correct or replace missing values where appropriate, and send malformed records to a separate error location for investigation rather than silently accepting incorrect data.

17. e) Explain how to monitor pipeline execution and validate data completeness.

- Use Azure Data Factory's monitoring features to check pipeline status, activity duration, failures, and error messages. Compare source and destination record counts, verify that expected files arrived, and check the transformed data for missing or invalid records.